Common Ground: Full Build Spec (UI + Backend)

A group-taste planning agent powered by Qloo. Built for the Qloo hackathon.

0. How to read this doc

Every feature below has the same four parts:

- **What it is** (plain words)
- **What the user sees** (UI work)
- **What the backend does** (server work)
- **Done when** (a simple test to know it works)

One important warning: I have not confirmed Qloo's exact endpoint names and parameters. Wherever this doc says "call Qloo," read the official Qloo API docs first and adapt. Design around what Qloo actually returns well.

1. The product in one paragraph

A group of 3 to 8 people (for example a family planning a Jaipur trip) each enters a few things they love. The agent turns those into Qloo entities, builds a taste profile for each person, finds where tastes overlap and who is the outlier, asks Qloo for dining, music, and activity options, scores every option for fairness, re-queries if someone is left out, and finally writes a plan. The writer is only allowed to mention things Qloo returned. The UI shows the whole thinking process live.

2. Recommended tech stack

Layer	Choice	Why
Frontend	Next.js (React) + Tailwind	Fast to build, easy to host on Vercel
Backend	Next.js API routes (Node)	One repo, one deploy
Agent brain	Claude API with tool use	Gives the multi-step loop without a heavy framework

Layer	Choice	Why
Taste data	Qloo API	The whole point of the hackathon
Streaming	Server-Sent Events (SSE)	Lets the UI show live agent steps
Cache	A JSON file or small KV store (Vercel KV / Upstash)	Presets never fail, quota is protected
Charts	D3 or React Flow for the overlap map	Node and bridge visual
Hosting	Vercel	Free, public URL, no login
License	MIT	Must be visible in the repo's About section

3. Pages and screens (UI overview)

Keep it to **three screens**. More screens means more things that can break.

1. **Home / Setup screen:** presets, group builder, plan type, city, constraints.
2. **Agent run screen:** live trace on one side, building results on the other.
3. **Result screen:** the final plan, overlap map, fairness bars, conflict card, LLM-vs-Qloo toggle.

On desktop, screens 2 and 3 can be one page that fills in as the agent works. On mobile, stack them vertically.

4. Backend overview

The backend has five parts:

1. **Qloo client:** a small wrapper that calls Qloo, handles errors, caches responses.
2. **Agent orchestrator:** the loop that decides which tool to call next (Claude with tool use).
3. **Scoring engine:** plain code (not AI) that computes overlap and fairness. This must be deterministic so results are explainable.
4. **Composer:** the final writing step, constrained to Qloo-returned entities.
5. **Infrastructure:** streaming, caching, rate limiting, secrets.

Suggested folder structure

```
common-ground/
  app/
    page.tsx                 # setup screen
    run/page.tsx             # agent run + result
    api/
      run/route.ts           # starts the agent, streams events (SSE)
      baseline/route.ts      # plain-LLM answer for the toggle
      presets/route.ts       # returns cached preset runs
  lib/
    qloo/client.ts           # Qloo wrapper + cache
    qloo/types.ts
    agent/orchestrator.ts    # the loop
    agent/tools.ts           # tool definitions
    scoring/profiles.ts      # taste profiles
    scoring/overlap.ts       # overlap + outliers
    scoring/fairness.ts      # fairness scoring
    scoring/constraints.ts   # hard filters
    compose/composer.ts      # final plan writer + grounding check
    compose/validate.ts      # checks no invented names
  components/                # UI pieces
  data/presets/              # cached example runs
  LICENSE
  README.md
```

5. Core features, one by one

Feature 1: Entity resolution with disambiguation

What it is. The person types "Rahman" or "Jab We Met." Qloo has its own ID for each real artist, film, restaurant, or brand. This step finds the right ID. Everything later depends on getting this right.

What the user sees (UI).

- A text box per love with live autocomplete (search as you type, after a short pause).
- Each suggestion shows a name, a type icon (music, film, food, brand), and a small subtitle (for example "Artist" or "2007 film") so similar names are easy to tell apart.
- Once chosen, the love becomes a small "chip" with a green tick meaning "matched."

- If a typed name matches nothing, show a gentle message: "We couldn't find this. Try another spelling or pick something similar."
- If two results are close, show a small "Did you mean…?" picker instead of silently choosing.

What the backend does.

- A search endpoint that sends the typed text to Qloo's search and returns the top few matches with ID, name, type, and any image or year.
- Debounce and cache results so typing does not burn quota.
- Store only the chosen Qloo ID plus display name in the person's profile.
- Handle failures: timeouts, empty results, rate limits.

Done when. You can type 5 messy names (misspelled, Hindi-English mixed, partial) and get the right entity or a clear "pick one" prompt every time.

Feature 2: Per-person taste profiles

What it is. A small "taste card" for each person built from their loves. It is the structured summary the rest of the system compares.

What the user sees (UI).

- After the agent resolves loves, each person gets a card: name, age band (optional), their chips, and 3 to 5 "taste tags" (for example classic, nostalgic, indie, high-energy).
- Tags have simple colors so you can spot similar people by color at a glance.
- Clicking a card expands it to show which of their loves produced which tags (this proves the profile is real, not invented).

What the backend does.

- For each resolved entity, ask Qloo for related information: tags or descriptors, and related or similar entities, wherever the API provides them.
- Merge a person's entities into one profile object, for example:

```
{
  "personId": "dadi",
  "entities": ["qloo_id_1", "qloo_id_2"],
  "tags": { "classic": 0.9, "nostalgic": 0.8, "traditional": 0.7 },
  "related": ["qloo_id_9", "qloo_id_14"],
  "constraints": { "vegetarian": true, "limitedWalking": true }
}
```

- Normalize tag weights (0 to 1) so people with more loves do not dominate.
- Keep the raw Qloo response next to the summary so the UI can explain it later.

Done when. Two very different people (for example a 72-year-old and a 15-year-old) produce clearly different tag sets.

Feature 3: Group overlap and outlier detection

What it is. Compare all the taste cards. Find what people share (overlap) and who shares the least (outlier).

What the user sees (UI).

- The **overlap map**: each person is a dot (node). Lines connect people who share tastes; thicker lines mean more overlap. Shared clusters appear as labeled bubbles ("classic Bollywood," "street food").
- The outlier is highlighted with a soft warning ring and a note: "Arjun shares the least with the group."
- A small "group core" list: the tags most people share.
- Hover or tap a line to see exactly which tags or entities create the connection.

What the backend does.

- Compute pairwise similarity between every two people (for example cosine similarity on tag-weight vectors, plus a bonus for shared entities or related entities).
- Compute each person's **average similarity to everyone else**. The lowest is the outlier.
- Find clusters (simple threshold grouping is enough for 3 to 8 people).
- Compute the "group core": tags present for at least 60% of people.
- Return a graph object (nodes, edges, clusters, outlier ID) that the UI can draw directly.

Done when. In a test group where one person is obviously different, that person is flagged as the outlier, and a reviewer can understand why from the map.

Feature 4: Taste-grounded queries across at least 3 domains

What it is. Ask Qloo for real recommendations in different areas (dining, music, venues or activities) using the group's blended taste as the signal.

What the user sees (UI).

- Result sections with clear headings: "Where to eat," "What to listen to," "What to do."
- Each result is a card with name, image if available, location, and a "match" score.

- A small label on every card: "From Qloo" so provenance is obvious.

What the backend does.

- Build a **blended signal**: the group core plus a weighted sample of each person's entities (give the outlier slightly extra weight so they are not drowned out).
- For each domain, call Qloo's recommendation or insights capability filtered by:
  - the entity type (restaurant, artist, place, and so on),
  - the city or location,
  - the blended signal.
- Also run **per-person queries** for the top candidates so you can later score each option for each person.
- Collect, dedupe, and store candidates with their source and raw Qloo data.
- Run the three domains in parallel to keep the wait short.

Done when. One run returns real, city-specific candidates in all three domains, and none of them were typed by you.

Feature 5: Fairness scoring

What it is. Every option gets a score for every person. Then we check the average and the worst score. An option that is great for five people and awful for one is a bad pick.

What the user sees (UI).

- **Fairness bars:** for each pick, one small bar per person, colored green to red, with the person's name or avatar.
- A single "fairness grade" per pick, for example "Fair for all" or "Leaves Arjun behind."
- A toggle to sort picks by "highest average" or "best for the least-happy person."
- A tooltip explaining the formula in one plain sentence.

What the backend does.

- For each candidate and each person, compute a fit score (0 to 100) from tag similarity, shared or related entities, and any per-person Qloo affinity data.
- Compute for each candidate:
  - `average` = mean of all scores,
  - `worst` = lowest score,
  - `fairness` = a blend, for example `0.5 * average + 0.5 * worst`, or `average - penalty if worst is under a threshold`.

- Rank by fairness, not by average alone.
- Mark a candidate as "needs repair" if any person falls under a threshold (for example 35).
- Keep this as plain deterministic code and log the numbers so the UI can show them exactly.

Done when. In a test, an option that is a 95 for four people and a 5 for one is ranked below a steady 70 for everyone.

Feature 6: The re-query loop

What it is. If the best plan still leaves someone unhappy, the agent goes back to Qloo and tries again, this time leaning toward that person. The user watches the plan improve.

What the user sees (UI).

- In the trace: "Arjun scored 12 on the top pick. Searching again with extra weight on his taste..."
- A before-and-after view: the first pick, the problem, the new pick, and the improvement ("Arjun: 12 to 64").
- A small badge on repaired picks: "Improved by re-search."

What the backend does.

- After scoring, check for anyone below the threshold.
- If found, build a new blended signal with that person's weight increased, query Qloo again, rescore.
- Limit to 2 or 3 rounds to avoid long waits and runaway cost.
- If still unresolved, stop and hand the case to the split-and-reunite logic and the conflict card.
- Record every round in the trace log with the numbers.

Done when. You can show a run where the first result had a low scorer and the final result is measurably better.

Feature 7: The LLM may name only entities Qloo returned

What it is. The AI writer can only mention restaurants, artists, and places that came from Qloo. This is your proof that Qloo is doing the real work and that the AI is not making things up.

What the user sees (UI).

- Every named place or artist in the final plan is a clickable tag. Clicking shows the Qloo source data behind it.
- A small "Verified from Qloo" indicator on the plan.
- If any claim fails the check, it is quietly removed or rewritten (the user should never see an invented name).

What the backend does.

- Give the composer a closed list of allowed entities (names and IDs) and instruct it to use only those.
- Ask for **structured output** (JSON): each plan item must reference an entity ID from the allowed list.
- Add a **validation step** after generation: scan the output, check that every named entity or ID is in the allowed set. If not, retry once with a stricter prompt, then drop the offending item.
- Keep generic words (like "morning walk") allowed, but require that any specific place or artist name has a matching ID.

Done when. You can deliberately try to make the model invent a restaurant and the validator catches it.

Feature 8: Hard constraints as filters

What it is. Some things are rules, not tastes: vegetarian, budget, mobility, allergies. They remove options completely instead of lowering a score.

What the user sees (UI).

- On each person's card, clear toggles or chips: Vegetarian, Vegan, Jain, No alcohol, Budget limit, Limited walking, Wheelchair access, Allergies.
- In results, a small note when an option was removed: "2 options hidden because they don't work for Dadi (vegetarian)."
- A "show hidden options" link so the system is transparent.

What the backend does.

- Keep constraints in a separate data structure from taste.
- Apply them **before** fairness scoring: any candidate that fails a *group-wide* hard rule is removed.

- For *per-person* constraints (for example only one person is non-veg), do not remove the option; instead require that it has a compatible choice for each person (for example the venue has both veg and non-veg options), or send it to the split plan.
- Use whatever fields Qloo provides (price level, tags, cuisine) and fall back to clearly labeled best-effort checks when data is missing. Never claim certainty you do not have.

Done when. A vegetarian-only test group never sees a steakhouse, regardless of how well the vibe matches.

6. Differentiator features, one by one

Feature 9: Split-and-reunite planning

What it is. Sometimes forcing everyone together makes everyone a little unhappy. The agent can split the group for part of the day and bring them back together.

What the user sees (UI).

- A day timeline with lanes: when the group splits, the timeline shows parallel lanes (for example "Dadi + Mummy" and "Riya + Arjun"), then they merge again at dinner.
- A short reason on each split: "Splitting here raises everyone's score from 48 to 76 on average."
- A switch: "Keep everyone together" vs "Allow splitting."

What the backend does.

- Try to partition the group into 2 subgroups (for 3 to 8 people, brute force all splits is cheap) that maximize each subgroup's fairness score.
- Compare: score of one shared activity vs score of two subgroup activities. Split only if the improvement passes a threshold (for example 15 points average, or fixes a person below the repair threshold).
- Always schedule at least one shared moment (usually dinner) so the group reunites.
- Account for practical limits (travel time between venues, mobility constraints) using basic location data.

Done when. The agent splits when it clearly helps and stays together when the group is already well-matched.

Feature 10: Honest conflict card

What it is. A clear box that says who is giving up what, and why.

What the user sees (UI).

- A card near the top of the result: "Heads up: not everyone gets everything."
- It lists each conflict in one line plus the compromise: "Chacha (non-veg) and Dadi (vegetarian): chosen venue has a full menu for both."
- Shows who is the "least-served" person and by how much.
- Tone matters: calm, specific, never preachy.

What the backend does.

- Generate conflicts from data, not from AI guessing: constraint clashes, people below the threshold in the final plan, outlier flagged earlier.
- Compute the "sacrifice" per person (their best possible score minus their score in the final plan).
- Pass those facts to the composer to phrase them kindly, but the facts themselves come from the scoring engine.

Done when. The card names real people and real numbers and matches the fairness bars exactly.

Feature 11: Per-pick "why this works for [names]"

What it is. A short, specific reason on every recommendation.

What the user sees (UI).

- Under each pick: one or two lines such as "Works for Dadi (classic Rajasthani food) and Arjun (casual, lively). Riya may find it less aesthetic."
- Small avatar chips showing who it works for, plus a lighter chip for who it works less well for.

What the backend does.

- Provide the composer with, for each pick, the top contributing tags and entities per person (from scoring).
- Require that each reason cites at least one shared tag or entity, and the validator checks it exists in the data.
- Include a "works less well for" note whenever someone scores under a threshold, so the tool stays honest.

Done when. Every reason can be traced to real data, and no two picks have copy-paste reasons.

Feature 12: Plain LLM vs Qloo-grounded toggle

What it is. A switch that shows the generic AI answer next to the Qloo-powered answer. This is your main proof.

What the user sees (UI).

- A tab or switch at the top of results: "Plain AI" and "Common Ground (Qloo)."
- Side-by-side on desktop, tabs on mobile.
- Highlighting: in the Qloo version, mark specifics (named places, per-person scores, split plans) in color; in the plain version, mark vague phrases ("try local cuisine").
- A short "what changed" summary beneath: "Qloo version: 7 specific places, 6 people scored, 1 repair round. Plain version: 0 scores, generic suggestions."

What the backend does.

- A separate endpoint that sends the **same group description** to the LLM **without** any Qloo data and asks for the same plan format.
- Use the same model and similar length so the comparison is fair.
- Compute simple comparison stats: number of named entities, number of per-person justifications, whether constraints were respected.
- Cache this result for presets.

Done when. A stranger looking at both versions can immediately tell which is more specific and useful.

Feature 13: Visible agent trace

What it is. A live log of each step the agent takes. It proves the system is really agentic.

What the user sees (UI).

- A panel showing steps appearing in real time with icons and short sentences:
  1. Resolving 18 loves… done
  2. Building 6 taste profiles… done
  3. Arjun is the outlier (similarity 0.21)
  4. Searching dining in Jaipur… 14 candidates
  5. Scoring fairness… top pick leaves Arjun at 12
  6. Re-searching with more weight on Arjun… improved to 64
  7. Writing the plan…
- Each step is expandable to show the tool call and result summary.

- A progress bar and an estimated time.

What the backend does.

- Stream events over SSE as the orchestrator works. Each event has: step name, status, short message, optional data.
- Log tool calls with inputs and trimmed outputs.
- Make sure steps are emitted as they happen, not all at the end.

Done when. The UI shows steps arriving live, and a judge can follow the agent's reasoning without reading code.

Feature 14: India-specific presets

What it is. One-click example groups with Indian tastes and situations. These also protect you if the live API is slow.

What the user sees (UI).

- Preset cards on the home screen, for example: "Sharma family trip to Jaipur (6 people)," "Shaadi planning: two families, one menu and playlist," "Friends' Diwali weekend," "Office offsite in Udaipur."
- Clicking a preset fills the form instantly and offers "Run live" or "See saved result."
- A short description for each so judges know what to expect.

What the backend does.

- Store preset inputs as JSON.
- Pre-run each preset against the real Qloo API and save the full result (trace, scores, plan, baseline) in `data/presets/`.
- Serve the saved result instantly; offer a live re-run behind a rate limit.
- Keep a note of when each preset was generated.

Done when. Each preset loads in under a second and the live re-run works too.

7. Visual features, one by one

Feature 15: Overlap map

What the user sees. Dots for people, lines for shared taste, bubbles for clusters, the outlier highlighted, and the final pick shown in the middle. Smooth animation as it builds during the run.

What the backend provides. A single JSON graph: `nodes`, `edges` (with weight and reason), `clusters`, `outlierId`, `recommendedPicks`.

Frontend notes. Use D3 force layout or React Flow. Keep labels readable on mobile. Provide a plain list view as fallback for small screens and accessibility.

Feature 16: Fairness bars

What the user sees. For every pick, one bar per person. Green (good) through yellow to red (poor). Name or initial beside each bar. Worst-matched person visibly marked.

What the backend provides. A matrix: picks by people, with scores and reasons.

Frontend notes. Never rely on color alone; add the number and a text label for accessibility.

Feature 17: Mobile-friendly layout

What the user sees. Everything works on a phone: single-column layout, large tap targets, collapsible panels for the trace, tabs instead of side-by-side comparison, no horizontal scrolling.

Frontend notes. Design mobile first. Test on a real phone and in browser device mode before submitting.

8. Polish and reliability features

Feature 18: No login, fast first result, progress display

- **UI:** no sign-in wall; the first meaningful content (the trace starting) appears within 2 seconds; a clear progress indicator throughout.
- **Backend:** stream early events immediately; run Qloo domain calls in parallel; keep each tool call under a timeout.

Feature 19: Rate limiting and quota protection

- **UI:** a friendly message if the limit is hit: "Live runs are busy. Try a saved example below." Always offer presets as the fallback.
- **Backend:** per-IP rate limit (for example 3 live runs per hour); global daily cap; cache every Qloo response by request key; never expose API keys to the browser.

Feature 20: Error handling

- **UI:** plain messages for: name not found, Qloo unavailable, LLM timeout, partial results. Never show raw errors or blank screens.
- **Backend:** retries with backoff, graceful degradation (if one domain fails, still return the others and say so), logging for debugging.

Feature 21: Accessibility and trust basics

- Readable contrast, keyboard navigation, alt text, numbers alongside colors.
- A short "How this works" panel and a note that results come from Qloo's taste data.
- A privacy line: no personal data stored; names entered are used only for the run.

9. The agent loop (how it all connects)

1. **Input arrives** (people, loves, constraints, city, plan type).
2. **Resolve entities** (Feature 1).
3. **Build profiles** (Feature 2).
4. **Compute overlap and outlier** (Feature 3).
5. **Apply hard constraints** (Feature 8).
6. **Query Qloo in 3+ domains** (Feature 4).
7. **Score fairness** (Feature 5).
8. **If someone is under threshold, re-query** (Feature 6), up to 2 or 3 rounds.
9. **Try split-and-reunite if still unresolved** (Feature 9).
10. **Build the conflict card from data** (Feature 10).
11. **Compose the plan with grounding rules** (Features 7 and 11).
12. **Validate** that every named entity came from Qloo.
13. **Generate the plain-LLM baseline** (Feature 12).
14. **Stream everything to the UI** (Feature 13).

Tools to give the Claude agent

Tool	Purpose
`search_entity`	Resolve a typed love to a Qloo entity

Tool	Purpose
`get_entity_profile`	Get tags and related entities
`get_recommendations`	Query Qloo by type, signal, and city
`score_candidates`	Run the deterministic fairness scoring
`requery_for_person`	Re-run with extra weight on one person
`propose_split`	Compute the best split into subgroups
`compose_plan`	Write the final plan from allowed entities only

Keep the math (scoring, overlap, splitting) in normal code that the tools call. Let the LLM decide *which tool next* and *how to phrase*, not how to calculate.

10. Data shapes to agree on early

```
// Input
{
  "planType": "family_trip",
  "city": "Jaipur",
  "days": 3,
  "people": [
    { "name": "Dadi", "age": 72, "loves": [], "constraints": ["vegetarian",
"limited_walking"] }
  ]
}

// Final result
{
  "trace": [],
  "profiles": [],
  "graph": { "nodes": [], "edges": [], "clusters": [], "outlierId": "" },
  "picks": [
    {
      "entityId": "",
      "name": "",
      "domain": "dining",
      "scores": { "dadi": 82 },
      "average": 0,
      "worst": 0,
```

```
      "why": "",
      "repaired": false
    }
  ],
  "itinerary": [],
  "conflicts": [],
  "baseline": { "text": "", "stats": {} }
}
```

11. Build order (suggested schedule)

1. **Day 1:** Read Qloo docs. Get search and one recommendation call working in a script. Save sample responses.
2. **Day 2:** Build profiles, overlap, and fairness code against the saved responses. Write tests.
3. **Day 3:** Build the agent loop with tool use and the SSE stream. Add re-query.
4. **Day 4:** Build the UI: setup screen, trace panel, results, fairness bars.
5. **Day 5:** Add overlap map, split-and-reunite, conflict card, comparison toggle.
6. **Day 6:** Presets, caching, rate limits, error handling, mobile pass.
7. **Day 7:** Deploy, test in incognito on a phone, write README, add LICENSE, final check.

If you have less time, cut in this order: split-and-reunite first, then the animated overlap map (use a simple list), then extra presets. Never cut entity resolution, fairness scoring, the re-query loop, grounding validation, or the comparison toggle.

12. Testing checklist before submitting

- Resolves misspelled and mixed-language names correctly
- Outlier is detected in a deliberately lopsided test group
- Fairness ranking prefers steady scores over one-person-unhappy scores
- A re-query visibly improves a low score
- Validator blocks an invented venue
- Vegetarian constraint removes non-veg-only venues
- Presets load instantly with no API call
- App survives Qloo being down (shows saved examples)
- Works on a phone, in incognito, with no login

- No API keys in the repo or browser network tab

13. Submission checklist

- **Live demo URL**, tested in an incognito window
- **Public repo** (GitHub) with all source, assets, and setup instructions
- **README** with: what it does, architecture diagram, env variables (`QLOO_API_KEY`, `ANTHROPIC_API_KEY`), how to run locally, how presets were generated
- **LICENSE** file (MIT or Apache-2.0) so it appears in the repo's About section
- **Project description** explaining the agent loop and exactly what makes it Qloo-powered (the grounding rule, the per-person scoring, the comparison toggle)

14. What would sink the project

- Plan text that reads like generic ChatGPT output
- The model naming places Qloo never returned
- A comparison toggle where both versions look the same
- A broken live demo or quota exhausted during judging
- Skipping the real Qloo data and faking scores

15. Final advice

The winning story is not "we built a recommender." It is: **"Here is a family whose tastes clash. Watch the agent find the person being ignored, search again for them, split the day when needed, and show exactly who gains and who compromises. A plain AI cannot do this because it does not know what these people actually love."** Build every feature in service of that one story, and test with real Qloo responses as early as possible.